

MisterFix

Plataforma de Serviços Domésticos

Disciplina	Lab. Desenvolvimento Aplicações Móveis e Distribuídas
Sprint	Sprint 1 — Proposta de Domínio e Backend REST
Entrega	11/05/2026
Tecnologias	Node.js + Express + SQLite + JWT
Repositório	https://github.com/matheustitton/MrFix

1. Descrição do Domínio

O MisterFix é uma plataforma digital que conecta pessoas que precisam de serviços de manutenção e reparos domésticos com profissionais autônomos das áreas de elétrica, hidráulica, alvenaria, pintura, marcenaria, entre outras. O nome une os conceitos de consertar e imediatismo, comunicando a proposta central da plataforma, resolução rápida de problemas do lar.

Uma das funcionalidades mais relevantes da plataforma é o filtro de gênero do prestador. Mulheres que vivem sozinhas ou que se sintam mais confortáveis recebendo uma profissional do gênero feminino em casa podem filtrar os resultados e solicitar exclusivamente prestadoras mulheres garantindo mais segurança e autonomia no processo.

O mercado de serviços domésticos no Brasil movimenta bilhões de reais por ano e conta com poucos intermediadores digitais confiáveis. Plataformas existentes como a GetNinjas oferece conexão entre clientes e profissionais, mas carece de funcionalidades voltadas à segurança da usuária. O MisterFix preenche essa lacuna ao incorporar, desde sua concepção, mecanismos de verificação e preferência de gênero como requisito funcional de primeira classe.

2. Funcionalidades de Usuário

2.1 Cliente

- Cadastro de perfil com dados pessoais e preferências de gênero do prestador;
- Buscar serviço por categoria (elétrica, hidráulica, pintura, etc.);
- Buscar prestador por especialidade, avaliação e gênero;
- Solicitar serviço com descrição detalhada, localização e agendamento;
- Receber token verificador ao início do serviço;
- Avaliar o prestador após a conclusão do serviço (1 a 5 estrelas);

- Realizar pagamento via cartão de débito, crédito ou PIX.

2.2 Prestador de Serviço

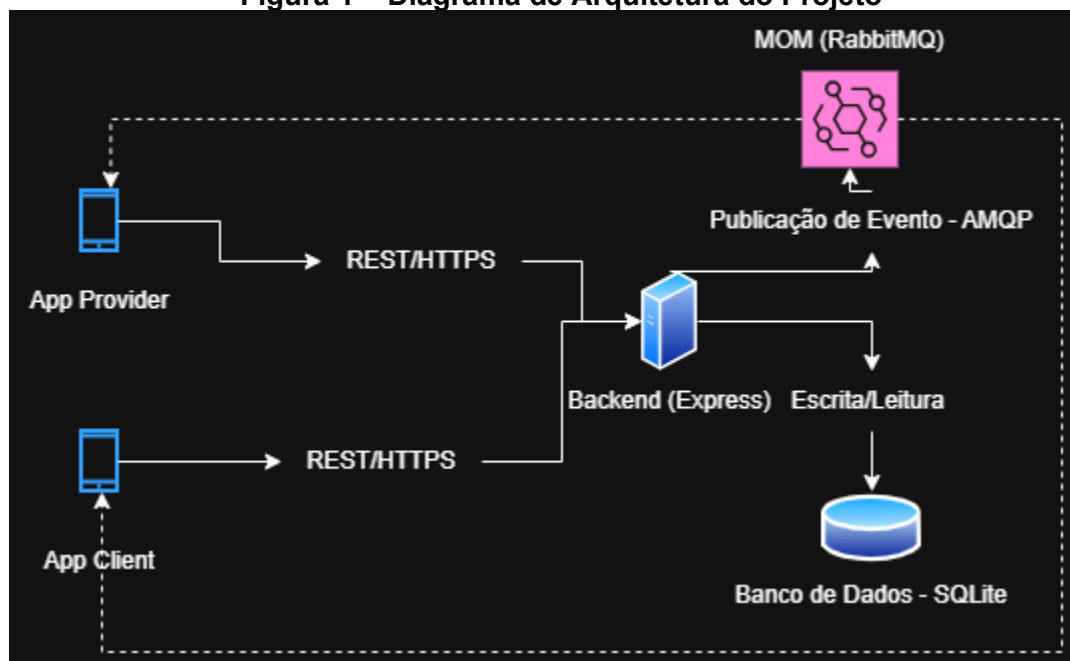
- Cadastrar perfil com especialidades, anos de experiência e preço médio por serviço;
- Receber notificações de novas solicitações compatíveis com seu perfil;
- Aceitar ou recusar solicitações pendentes;
- Validar o token verificador do cliente ao iniciar o serviço;
- Atualizar o status da solicitação;
- Avaliar o cliente após a conclusão do serviço.

3. Visão Geral da Arquitetura

O sistema adota uma Arquitetura Orientada a Eventos (EDA) com os seguintes componentes:

Componente	Tecnologia	Responsabilidade
App Cliente	Flutter/Dart	Interface do usuário final para solicitar serviços
App Prestador	Flutter/Dart	Interface do profissional para aceitar e executar serviços
Backend REST	Node.js + Express	API RESTful com lógica de negócio e persistência
Banco de Dados	SQLite (Sequelize ORM)	Persistência de usuários, solicitações e avaliações
MOM	RabbitMQ	Comunicação assíncrona entre backend e apps

Figura 1 – Diagrama de Arquitetura do Projeto



4. Endpoints Implementados — Sprint 1

Método	Rota	Descrição	Auth
POST	/api/auth/register	Cadastro de usuário (cliente ou prestador)	Não
POST	/api/auth/login	Autenticação e geração de token JWT	Não
GET	/api/auth/me	Dados do usuário autenticado	JWT
GET	/api/categories	Listar categorias de serviço	Não
GET	/api/providers	Listar prestadores (filtro: gênero, categoria)	Não
GET	/api/providers/:id	Perfil completo do prestador	Não
POST	/api/providers/specialties	Prestador adiciona especialidade e preço	JWT
POST	/api/service-requests	Cliente cria solicitação de serviço	JWT
GET	/api/service-requests	Listar solicitações (por perfil do usuário)	JWT
GET	/api/service-requests/:id	Detalhe de solicitação	JWT
PATCH	/api/service-requests/:id/status	Atualizar status	JWT
POST	/api/ratings	Avaliar após conclusão do serviço	JWT
GET	/api/ratings/user/:userId	Avaliações recebidas por um usuário	Não